TASK2 数据炼金术：数据诊断与构造交易指标

在量化交易领域，数据的质量和深度决定了交易策略的有效性和精准性。本次任务旨在通过构造关键交易指标来捕捉市场动态和交易信号，为后续策略开发提供坚实的数据基础。

任务 1. 对数据进行基础的诊断分析，检查缺失值，计算描述性统计量。

针对原始数据的各个字段，可以设计以下统计量。

表 1　统计量设计

字段类别	字段	分析内容
标识	ts_code，trade_date	日期连续性
价格	open，high，low，close，pre_close	描述性统计
变动	change，pct_chg	分布，正负天数比
数量	vol，amount	分布，异常检测

以下是 Py 代码与对应的诊断分析结果：

1. 缺失值

```
def check_missing(df: pd.DataFrame) -> pd.DataFrame:
    """检查每列的缺失值情况。返回缺失值汇总表。"""
    missing = pd.DataFrame({
        "column": df.columns,
        "missing_count": df.isnull().sum().values,
        "missing_pct": (df.isnull().sum() / len(df) *
100).round(3).values,
    })
    missing = missing[missing["missing_count"] >
0].reset_index(drop=True)

    if missing.empty: print("所有列均无缺失值。")
    else: print("发现缺失值：")
    for _, row in missing.iterrows():
        print(f" {row['column']:12s} — {row['missing_count']:4d} 个
缺失 ({row['missing_pct']:.2f}%)")
    return missing
```

结果表明，全部数据 243 行 11 列无缺失值。

2. 日期连续性

```
def check_date_continuity(df: pd.DataFrame) -> None:
    """检查交易日期的连续性和覆盖范围。"""
    dates = df["trade_date"].sort_values()
    n_days = len(dates)
    date_range = (dates.iloc[-1] - dates.iloc[0]).days
    weekdays = sum(1 for d in dates if d.dayofweek < 5)

    # 自然日 vs 实际交易日
    print(f" 日期范围: {dates.iloc[0].strftime('%Y-%m-%d')} →
{dates.iloc[-1].strftime('%Y-%m-%d')}")
    print(f" 自然日跨度: {date_range} 天")
    print(f" 实际交易日: {n_days} 天")
    print(f" 其中工作日: {weekdays} 天")
    print(f" 数据覆盖率: {n_days / date_range * 100:.1f}% (自然日)")

    # 检查是否有重复日期
    dup_dates = dates[dates.duplicated()]
    if len(dup_dates): print(f"重复日期: {len(dup_dates)} 条")
    else: print(f"无重复日期")

    # 检查期间内是否有休市日缺失（粗略：周末除外）
    expected_trading_days = len(pd.bdate_range(dates.iloc[0],
dates.iloc[-1]))
    gap = expected_trading_days - n_days
    if gap > 0:
        print(f"相比工作日历少 {gap} 天（含法定节假日休市，属正常现象）")
```

结果表明，自然日跨度 365 天，交易日 243 天，无重复，缺少工作日均为法定节假日。

3. 描述性统计

```
def descriptive_stats(df: pd.DataFrame) -> pd.DataFrame:
    """计算所有数值列的扩展描述性统计量。"""
    data = df[ALL_NUMERIC].copy()
    stats = pd.DataFrame(index=ALL_NUMERIC)
```

```
    # 基本统计量
    stats["count"] = data.count()
    stats["mean"] = data.mean().round(4)
    stats["median"] = data.median().round(4)
    stats["std"] = data.std().round(4)
    stats["min"] = data.min().round(4)
    stats["max"] = data.max().round(4)
    stats["range"] = (data.max() - data.min()).round(4)
    stats["Q1"] = data.quantile(0.25).round(4)
    stats["Q3"] = data.quantile(0.75).round(4)
    stats["IQR"] = (stats["Q3"] - stats["Q1"]).round(4)

    # 分布形态
    stats["skewness"] = data.skew().round(4)
    stats["kurtosis"] = data.kurtosis().round(4)
    if "change" in data.columns:
        stats.loc["change", "positive_days"] = (data["change"] >
0).sum()
        stats.loc["change", "negative_days"] = (data["change"] <
0).sum()
        stats.loc["change", "zero_days"] = (data["change"] ==
0).sum()

    # 变异系数 CV = std/|mean| (仅当 mean ≠ 0)
    stats["CV"] = (data.std() / data.mean().abs()).round(4)
    return stats
```

表 2　针对价格变量的描述性统计（单位：元）

统计量	open	high	low	close	pre_close
均值	1197.94	1240.30	1169.76	1206.48	1200.31
中位数	1295.00	1329.87	1262.69	1299.20	1287.00
标准差	266.55	278.00	257.04	268.11	266.90
最小	523.00	537.87	520.67	523.50	523.50
最大	1824.94	1966.00	1792.80	1868.00	1825.00
偏度	-1.16	-1.10	-1.17	-1.10	-1.16
峰度	0.72	0.80	0.78	0.78	0.68

从结果来看，价格整体呈左偏，说明多数时候价格处于高位，偶有大幅回调。峰度为正数，表明分布比正态分布更加尖峰厚尾。

表 3　针对变动变量的描述性统计

统计量	change（元）	pct_chg（%）
均值	+6.17	+0.64%
标准差	56.30	4.60%
最大涨幅	+283.33	+20.00%
最大跌幅	-203.00	-14.45%
偏度	+0.95	+1.17

针对变动变量描述性统计的结果表明，涨跌幅呈右偏分布，正收益的尾部比负收益更厚，一年内出现了多次大幅上涨。

表 4　针对变动变量的描述性统计

统计量	vol（手）	amount（千元）
均值	100，412	12，460，160
中位数	90，298	10，508，390
最大值	231，722	36，853，370
偏度	+0.86	+0.85

4. 异常检测

```python
def detect_outliers_iqr(df: pd.DataFrame, cols: list, multiplier:
float = 1.5) -> pd.Series:
    """用 IQR 方法检测异常值，返回每列的异常值数量。"""
    outliers = {}
    for col in cols:
        q1 = df[col].quantile(0.25)
        q3 = df[col].quantile(0.75)
        iqr = q3 - q1
        lower = q1 - multiplier * iqr
        upper = q3 + multiplier * iqr
        n_out = ((df[col] < lower) | (df[col] > upper)).sum()
        outliers[col] = n_out
```

```
return pd.Series(outliers, name="outlier_count")
```

表 5　异常检测

变量	异常数	占比	评估（5%）
open/high/low/close	21-25	~9%	异常
change/pct_chg	9-12	~4%	正常
vol/amount	2-4	~1%	正常

如公式 1 所示，采用 IQR 方法，分别定义上下边界 *Lower*、*Upper*，认为在该数据之外的数据是异常值。其中，Q_1 和 Q_3 分别是第一四分位数与第三四分位数。结果显示，每个价格变量都有 9%左右的异常数据；其余数据在 IQR 规则下没有明显异常。

$$\begin{aligned} IQR &= Q_3 - Q_1, \\ Lower &= Q_1 - 1.5 \times IQR, \\ Upper &= Q_3 + 1.5 \times IQR. \end{aligned} \tag{1}$$

5. 涨跌分布

数据显示，股价累计上涨天数 131 天，占比 53.9%；下跌天数 111 天，占比 45.7%；持平天数 1 天，占比 0.4%。平均涨幅+3.62%，平均跌幅-2.87%，盈亏比为 1.26。这一结果表明，寒武纪过去一年的整体表现较强劲，上涨天数多于下跌，且平均涨幅高于平均跌幅，属于高波动成长股的特征。

任务 2. 了解 RSI，MACD，布林带（Bollinger Bands），给出指标计算方法及作用。

下面按照提出指标的初衷、计算方法、结果解读以及指标作用分别阐述。

1. 相对强弱指数 RSI (Relative Strength Index)

【提出指标的初衷】 RSI 由 J. Welles Wilder Jr. 于 1978 年提出，其设计初衷是衡量股票在一段时间内上涨力量与下跌力量的相对强弱，从而判断市场是否已经进入过度上涨或过度下跌的状态。传统的价格分析通常只能观察股价涨跌的幅度，却难以回答“当前上涨是否已经过度”这一问题。Wilder 认为，当市场连续上涨时，虽然价格仍在创新高，但买方力量可能已经逐渐衰竭；反之，连续下跌后卖方力量也可能逐渐耗尽。因此，他设计了 RSI 指标，通过比较一定时期内平均上涨幅度与平均下跌幅度，将市场动量映射到 0～100 的区间，为判断市场情绪和价格动能提供量化依据。

【计算方法】 RSI 的计算通常采用 14 个交易日作为统计窗口。首先分别计算过去 14 日内的平均上涨幅度（*AverageGain*）和平均下跌幅度（*AverageLoss*），然后计算两者的比值 RS，最后将其映射到 0～100 的区间，计算公式如下：

$$RS=\frac{AverageGain}{AverageLoss},\qquad RSI=100-\frac{100}{1+RS} \tag{2}$$

【结果解读】 RSI 的取值范围为 0 至 100。当 RSI 接近 50 时，说明近期上涨力量与下跌力量较为均衡，市场处于相对平稳状态；当 RSI 持续升高时，表明上涨动能不断增强；当 RSI 持续降低时，则说明下跌动能不断增强。一般认为，当 RSI 高于 70 时，市场可能已经进入超买（Overbought）状态，短期继续上涨的空间有限；当 RSI 低于 30 时，则可能进入超卖（Oversold）状态，股价存在反弹的可能性。

【指标作用】 RSI 主要用于衡量市场动量（Momentum），帮助投资者判断价格走势是否已经偏离正常水平。在震荡行情中，RSI 对识别超买和超卖区域具有较高的参考价值，可辅助寻找买入和卖出时机。此外，RSI 还常用于分析价格与指标之间的“背离”现象，即价格继续创新高（或新低），而 RSI 却未同步创新高（或新低），这种情况通常意味着当前趋势的动能正在减弱，可能预示着趋势即将发生反转。

2. 指数平滑异同移动平均线 MACD (Moving Average Convergence Divergence)

【提出指标的初衷】 MACD 由 Gerald Appel 于 20 世纪 70 年代提出，其设计目标是通过比较短期趋势与长期趋势之间的差异，帮助投资者识别趋势的形成、延续以及反转。传统移动平均线虽然能够反映价格趋势，但通常存在一定的滞后性，而且多条均线之间的交叉关系较为复杂，不易分析。MACD 将不同周期指数移动平均线（EMA）之间的距离进行量化，并进一步计算信号线，使投资者能够更加直观地观察趋势强弱及其变化过程。

【计算方法】 MACD 首先分别计算股票价格的短期指数移动平均线（通常为 12 日 EMA）和长期指数移动平均线（通常为 26 日 EMA），两者之差记为 DIF；随后再计算 DIF 的 9 日指数移动平均线作为信号线 DEA；最后，MACD 柱状图由 DIF 与 DEA 的差值得到。其计算公式可表示为：

$$\mathrm{DIF}=\mathrm{EMA}_{12}-\mathrm{EMA}_{26},\qquad \mathrm{DEA}=\mathrm{EMA}(\mathrm{DIF},9),\qquad \mathrm{MACD}=\mathrm{DIF}-\mathrm{DEA} \tag{3}$$

【结果解读】 MACD 的核心在于观察 DIF 与 DEA 两条曲线之间的相对位置及 MACD 柱状图的变化。当 DIF 位于 DEA 上方时，通常表示短期上涨趋势强于长期趋势，市场处于较强走势；反之，当 DIF 位于 DEA 下方时，则说明市场整体偏弱。柱状图越高，说明上涨动能越强；柱状图逐渐缩短，则表示当前趋势动能正在减弱。当 DIF 向上突破 DE

A 时，称为“金叉”，通常被视为看涨信号；当 DIF 向下跌破 DEA 时，称为“死叉”，通常被视为看跌信号。

【指标作用】 MACD 主要用于识别市场趋势及趋势变化方向。在单边上涨或下跌行情中，MACD 能较好地反映趋势是否已经形成以及趋势是否仍在延续，因此广泛应用于中长期趋势分析。此外，当价格不断创新高（或新低），而 MACD 未同步创新高（或新低）时，也会形成顶背离或底背离现象，这通常意味着当前趋势正在减弱，具有一定的趋势反转预警作用。

3. 布林带 Bollinger Bands

【提出指标的初衷】 布林带由 John Bollinger 于 20 世纪 80 年代提出，其设计初衷是在移动平均线的基础上进一步刻画价格波动范围。传统均线只能反映价格的平均水平，却无法判断当前价格距离正常波动范围有多远。John Bollinger 将统计学中的标准差引入技术分析，通过在移动平均线两侧构建上下轨道，使指标不仅能够反映价格趋势，还能够反映市场波动率（Volatility）的变化，从而帮助投资者判断当前价格是否偏离正常水平。

【计算方法】 布林带通常采用 20 日简单移动平均线（SMA）作为中轨，并利用过去 20 日收盘价计算标准差 σ。上轨由中轨加上两倍标准差得到，下轨由中轨减去两倍标准差得到，其计算公式如下：

$$\text{Middle} = \text{SMA}_{20}, \qquad \text{Upper} = \text{SMA}_{20}+2\sigma, \qquad \text{Lower} = \text{SMA}_{20}-2\sigma \tag{4}$$

【结果解读】 布林带由上轨、中轨和下轨三条曲线组成。中轨表示近期价格的平均水平；上轨和下轨分别表示价格正常波动范围的上界和下界。当股价接近或突破上轨时，说明当前价格相对于近期平均水平偏高；当股价接近或突破下轨时，则说明当前价格相对偏低。

此外，布林带的宽度能够反映市场波动率：当上下轨逐渐扩张时，说明市场波动加剧；当上下轨逐渐收窄时，则说明市场波动减弱。

【指标作用】 布林带主要用于衡量价格波动率，并辅助判断当前价格是否偏离正常波动区间。在震荡行情中，价格接近上轨往往意味着市场已经处于相对高位，而接近下轨则意味着市场处于相对低位，具有一定的均值回归参考价值。在趋势行情中，股价可能会沿着上轨或下轨持续运行，因此布林带通常需要与其他趋势指标（如 MACD）配合使用。此外，当布林带出现明显“收口”（Band Squeeze）时，意味着市场波动率处于较低水平，通常预示着未来可能出现较大的价格波动，但无法仅依据布林带判断突破方向。

任务 3. python 编程实现以下内容

1）计算 RSI，MACD，布林带 (Bollinger Bands)

Py 代码：

```
def compute_rsi(close: pd.Series, period: int = RSI_PERIOD) ->
pd.Series:
    """
    计算相对强弱指数 RSI (Wilder's smoothing)。
    RSI = 100 - 100 / (1 + RS)
    其中 RS = 平均涨幅 / 平均跌幅
    """
    delta = close.diff()
    gain = delta.clip(lower=0)
    loss = (-delta).clip(lower=0)

    # Wilder's EMA: 初始值为简单平均，后续用平滑公式
    avg_gain = gain.ewm(alpha=1 / period, min_periods=period,
adjust=False).mean()
    avg_loss = loss.ewm(alpha=1 / period, min_periods=period,
adjust=False).mean()

    rs = avg_gain / avg_loss.replace(0, np.nan)
    rsi = 100.0 - (100.0 / (1.0 + rs))
    rsi.name = "RSI"
    return rsi

def compute_macd(close: pd.Series) -> pd.DataFrame:
    """
    计算 MACD 指标。
    返回 DataFrame:
        DIF — 快慢 EMA 差值 (EMA_fast - EMA_slow)
        DEA — DIF 的信号线 (EMA of DIF)
        MACD — 柱状图 = 2 × (DIF - DEA)
    """
    ema_fast = close.ewm(span=MACD_FAST, adjust=False).mean()
    ema_slow = close.ewm(span=MACD_SLOW, adjust=False).mean()
    dif = ema_fast - ema_slow
    dif.name = "DIF"
```

```python
    dea = dif.ewm(span=MACD_SIGNAL, adjust=False).mean()
    dea.name = "DEA"
    macd_hist = 2.0 * (dif - dea) # 国内常用 ×2 柱
    macd_hist.name = "MACD"
    return pd.concat([dif, dea, macd_hist], axis=1)

def compute_bollinger(close: pd.Series) -> pd.DataFrame:
    """
    计算布林带 Bollinger Bands。

    返回 DataFrame:
        BB_MID — 中轨 (SMA)
        BB_UPPER — 上轨 (中轨 + K × σ)
        BB_LOWER — 下轨 (中轨 - K × σ)
        BB_WIDTH — 带宽百分比 = (上轨 - 下轨) / 中轨 × 100
    """
    mid = close.rolling(window=BB_PERIOD).mean()
    std = close.rolling(window=BB_PERIOD).std()
    upper = mid + BB_STD * std
    lower = mid - BB_STD * std
    width = (upper - lower) / mid * 100

    return pd.DataFrame({
        "BB_MID": mid,
        "BB_UPPER": upper,
        "BB_LOWER": lower,
        "BB_WIDTH": width,
    }, index=close.index)
```

运行结果:

```
============================================================
  📡 最新交易日信号摘要
============================================================
  日期: 2026-07-01
  收盘价: 1485.33 元

  RSI(14):      中性 (55.7)
  MACD:           多头金叉区域 📈
    DIF=53.62  DEA=35.15  Hist=36.96
  布林带:         通道内 (74% 带宽)
    Upper=1599.56  Mid=1378.27  Lower=1156.98
  MA 排列:        多头排列但价在均线下方
============================================================
```

图 1　指标摘要

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
1	trade_date	close	MA5	MA10	MA20	MA60	MA120	RSI	DIF	DEA	MACD	BB_MID	BB_UPPER	BB_LOWE	BB_WIDTH
2	2025/7/1	563							0	0	0				
3	2025/7/2	545.1							-1.4279	-0.2856	-2.2847				
4	2025/7/3	547.3							-2.3549	-0.6994	-3.3109				
5	2025/7/4	547.47							-3.0408	-1.1677	-3.7461				
6	2025/7/7	541.38	548.85						-4.0293	-1.74	-4.5785				
7	2025/7/8	542.77	544.804						-4.6469	-2.3214	-4.6511				
8	2025/7/9	535	542.784						-5.6977	-2.9967	-5.4022				
9	2025/7/10	523.5	538.024						-7.3735	-3.872	-7.0029				
10	2025/7/11	553.98	539.326						-6.1709	-4.3318	-3.6782				
11	2025/7/14	565.06	544.062	546.456					-4.2745	-4.3203	0.0917				
12	2025/7/15	553.1	546.128	545.466					-3.6941	-4.1951	1.002				
13	2025/7/16	580.19	555.166	548.975					-1.0362	-3.5633	5.0542				
14	2025/7/17	582.4	566.946	552.485					1.2343	-2.6038	7.6761				
15	2025/7/18	582.62	572.674	556					3.0166	-1.4797	8.9927				
16	2025/7/21	582	576.062	560.062				30.6529	4.3292	-0.3179	9.2943				
17	2025/7/22	593.93	584.228	565.178				35.4764	6.2599	0.9976	10.5246				
18	2025/7/23	595.56	587.302	571.234				36.1301	7.8313	2.3644	10.9339				
19	2025/7/24	600.23	590.868	578.907				38.066	9.3457	3.7607	11.1702				
20	2025/7/25	673.3	609.004	590.839				59.0041	16.2547	6.2595	19.9905				
21	2025/7/28	679	628.404	602.233	574.3445			60.1363	21.9371	9.395	25.0843	574.3445	656.5272	492.1618	28.6179
22	2025/7/29	710.78	651.774	618.001	581.7335			65.8062	28.6744	13.2509	30.847	581.7335	683.7917	479.6753	35.0876
23	2025/7/30	684.7	669.602	628.452	588.7135			58.458	31.5456	16.9098	29.2716	588.7135	698.987	478.44	37.4635

图 2　指标计算结果保存本地

2）根据指标计算结果，绘制对应可视化图形

Py 代码：

```
def plot_dashboard(df_raw: pd.DataFrame, indicators: pd.DataFrame,
ts_code: str) -> None:
    """
    绘制技术指标综合看板，包含四个面板：
    Panel A — K线（收盘价） + 布林带 + 多周期 MA
    Panel B — RSI（含超买超卖区域）
    Panel C — MACD（DIF / DEA / 柱状图）
    """
    close = indicators["close"]
    n = len(close)

    # 布局
    fig = plt.figure(figsize=(20, 13))
    gs = fig.add_gridspec(3, 1, height_ratios=[2.5, 1, 1],
hspace=0.12)

    ax_price = fig.add_subplot(gs[0])
    ax_rsi = fig.add_subplot(gs[1], sharex=ax_price)
    ax_macd = fig.add_subplot(gs[2], sharex=ax_price)

    date_range_str = (f"{close.index[0].strftime('%Y-%m-%d')} —
"f"{close.index[-1].strftime('%Y-%m-%d')}")
    fig.suptitle(f"{ts_code} 寒武纪 · 技术指标看板 | {date_range_str} |
```

```
{n} 个交易日", fontsize=15, fontweight="bold", y=0.98, )

    # Panel A: 价格 + 布林带 + MA
    ax_price.fill_between(
        indicators.index,
        indicators["BB_UPPER"], indicators["BB_LOWER"],
        color="steelblue", alpha=0.08, label="Bollinger Band (±2σ)",
    )
    ax_price.plot(
        indicators.index,
        indicators["BB_UPPER"],
        color="steelblue", linewidth=0.6, alpha=0.7, linestyle="--"
    )
    ax_price.plot(
        indicators.index,
        indicators["BB_MID"],
        color="steelblue", linewidth=0.6, alpha=0.9, linestyle="-."
    )
    ax_price.plot(
        indicators.index,
        indicators["BB_LOWER"],
        color="steelblue", linewidth=0.6, alpha=0.7, linestyle="--"
    )

    # MA 线
    for i, p in enumerate(MA_PERIODS):
        col = f"MA{p}"
        ax_price.plot(
            indicators.index, indicators[col],
            color=MA_COLORS[i], linewidth=MA_WIDTHS[i],
            alpha=MA_ALPHAS[i], label=f"MA{p}"
        )

    # 收盘价（最后画，在最上层）
    ax_price.plot(
        indicators.index, close,
        color="#1f2937", linewidth=1.2, label="Close", zorder=3
    )
```

```
    ax_price.set_ylabel("Price (CNY)", fontsize=10)
    ax_price.legend(loc="upper left", ncol=8, fontsize=7.5,
framealpha=0.6, edgecolor="gray")
    ax_price.grid(True, linestyle="--", alpha=0.25)
    ax_price.tick_params(labelbottom=False)

    #  Panel B: RSI
    ax_rsi.fill_between(indicators.index, RSI_OVERBOUGHT, 100,
color="red", alpha=0.06)
    ax_rsi.fill_between(indicators.index, 0, RSI_OVERSOLD,
color="green", alpha=0.06)
    ax_rsi.axhline(RSI_OVERBOUGHT, color="red", linewidth=0.8,
linestyle="--", alpha=0.5, label=f"Overbought ({RSI_OVERBOUGHT})")
    ax_rsi.axhline(50, color="gray", linewidth=0.5, alpha=0.4)
    ax_rsi.axhline(RSI_OVERSOLD, color="green", linewidth=0.8,
linestyle="--", alpha=0.5, label=f"Oversold ({RSI_OVERSOLD})")

    ax_rsi.plot(indicators.index, indicators["RSI"],
color="#6d28d9", linewidth=0.9, label=f"RSI({RSI_PERIOD})")
    ax_rsi.fill_between(
        indicators.index, 50, indicators["RSI"],
        where=(indicators["RSI"] >= 50),
        color="#6d28d9", alpha=0.08
    )
    ax_rsi.fill_between(
        indicators.index, indicators["RSI"], 50,
        where=(indicators["RSI"] < 50),
        color="#dc2626", alpha=0.08
    )

    ax_rsi.set_ylabel("RSI", fontsize=10)
    ax_rsi.set_ylim(0, 100)
    ax_rsi.set_yticks([0, 30, 50, 70, 100])
    ax_rsi.legend(loc="upper left", fontsize=7.5, framealpha=0.6)
    ax_rsi.grid(True, linestyle="--", alpha=0.25)
    ax_rsi.tick_params(labelbottom=False)
```

```
    # Panel C: MACD
    dif = indicators["DIF"]
    dea = indicators["DEA"]
    macd_hist = indicators["MACD"]

    # 柱状图 — 涨红跌绿
    colors_hist = ["#dc2626" if v >= 0 else "#16a34a" for v in macd_hist]
    ax_macd.bar(
        indicators.index, macd_hist, width=0.8,
        color=colors_hist, alpha=0.5, label="MACD Histogram"
    )

    ax_macd.plot(indicators.index, dif, color="#2563eb",
linewidth=0.9, label=f"DIF (EMA{MACD_FAST}-EMA{MACD_SLOW})")
    ax_macd.plot(indicators.index, dea, color="#ea580c",
linewidth=0.9, label=f"DEA (EMA{MACD_SIGNAL})")
    ax_macd.axhline(0, color="gray", linewidth=0.5, alpha=0.4)

    ax_macd.set_ylabel("MACD", fontsize=10)
    ax_macd.legend(loc="upper left", fontsize=7.5, framealpha=0.6)
    ax_macd.grid(True, linestyle="--", alpha=0.25)

    #  X 轴格式化
    for ax in [ax_price, ax_rsi, ax_macd]:
        ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y-%m"))
        ax.xaxis.set_major_locator(mdates.MonthLocator(interval=1))
        plt.setp(
            ax_macd.get_xticklabels(), rotation=45,
            ha="right", fontsize=8
        )
        plt.setp(
            ax_price.get_xticklabels(), rotation=45,
            ha="right", fontsize=8
        )
        plt.setp(
            ax_rsi.get_xticklabels(), rotation=45,
            ha="right", fontsize=8
        )
```

```
    fig.tight_layout(rect=[0, 0, 1, 0.96])

    # 保存
    output_dir = os.path.join(os.path.dirname(__file__), "..",
"output", "figures")
    os.makedirs(output_dir, exist_ok=True)
    output_path = os.path.join(output_dir,
"indicators_dashboard.png")
    fig.savefig(output_path, dpi=180, bbox_inches="tight",
facecolor="white", edgecolor="none")
    print(f"  技术指标看板已保存至: {output_path}")
    plt.close(fig)
```

运行结果:

图 3　技术指标看板

任务 4. 扩展了解还有其他哪些典型指标，选取一个进行介绍和计算，并用图形表示

MACD 和布林带都是在移动平均线（MA，Moving Average）的基础上改进得来，因此这里选择移动平均线进行扩展了解与对比。下面会梳理移动平均线提出的初衷、计算方法、结果解读以及指标作用，然后会在获取的寒武纪数据上进行计算和图形展示。

【提出指标的初衷】 移动平均线 MA 是技术分析中较基础的趋势分析工具之一，其设计初衷是通过对一段时间内的股票价格进行平均，消除短期价格波动带来的噪声，从而更加清晰地反映市场的整体运行趋势。股票价格在日常交易过程中会受到市场情绪、消息面以及随机交易行为等因素的影响，短期内往往呈现出较大的波动，仅观察每日价格容易受到偶然因素干扰，难以准确判断市场的真实走势。移动平均线通过不断更新固定时间窗口内的平均价格，使曲线更加平滑，帮助投资者识别市场当前处于上涨趋势、下跌趋势还是震荡趋势，也成为后来众多技术指标（如 MACD、布林带等）的理论基础。

【计算方法】 简单移动平均线会计算最近 N 个交易日收盘价的算术平均值，并随着时间推移不断更新统计窗口。常见的周期包括 5 日、10 日、20 日、60 日和 120 日等，不同周期分别反映短期、中期和长期的价格趋势。其计算公式如下：

$$\mathrm{SMA_N} = \frac{1}{\mathrm{N}}\sum_{\mathrm{i=0}}^{\mathrm{N-1}} \mathrm{P_{t-i}} \tag{5}$$

其中，P_{t-i} 表示第 $t-i$ 个交易日的收盘价，N 表示移动平均线所采用的统计周期。

【结果解读】 移动平均线反映的是股票在一定时间内的平均价格水平，其斜率和位置能够体现市场趋势的方向。当移动平均线持续向上倾斜时，说明近期平均价格不断提高，市场整体处于上涨趋势；当移动平均线持续向下倾斜时，则说明市场整体处于下跌趋势；当移动平均线基本保持水平时，则通常意味着市场处于震荡整理阶段。

此外，股价与移动平均线之间的相对位置也具有一定的参考意义。当股价长期运行在移动平均线上方时，通常说明市场整体较强；反之，当股价长期运行在移动平均线下方时，则说明市场整体偏弱。实际分析中，不同周期移动平均线之间的交叉关系也经常用于辅助判断趋势变化，例如短周期均线向上突破长周期均线通常被称为“金叉”，而向下跌破则称为“死叉”。

【指标作用】 移动平均线是一种典型的趋势跟踪指标，其主要作用是识别市场趋势、过滤短期价格噪声以及辅助判断买卖时机。在趋势明显的市场中，移动平均线能够帮助投资者顺应市场方向，避免受到短期价格波动的干扰。同时，不同周期移动平均线之间的排列关系能够反映市场趋势的强弱，例如短期均线位于长期均线上方且多条均线依次向上排列时，通常

表明市场处于较强的上升趋势。由于移动平均线本质上属于历史价格的平均值，因此具有一定的滞后性，对于趋势反转的反应通常慢于价格本身，这也是后来指数移动平均线（EMA）、MACD 等指标在其基础上进一步发展的重要原因。

针对寒武纪交易数据的移动平均线计算和绘制已经集成在上一个任务中一并完成了，具体见图 3 的第一个子图。